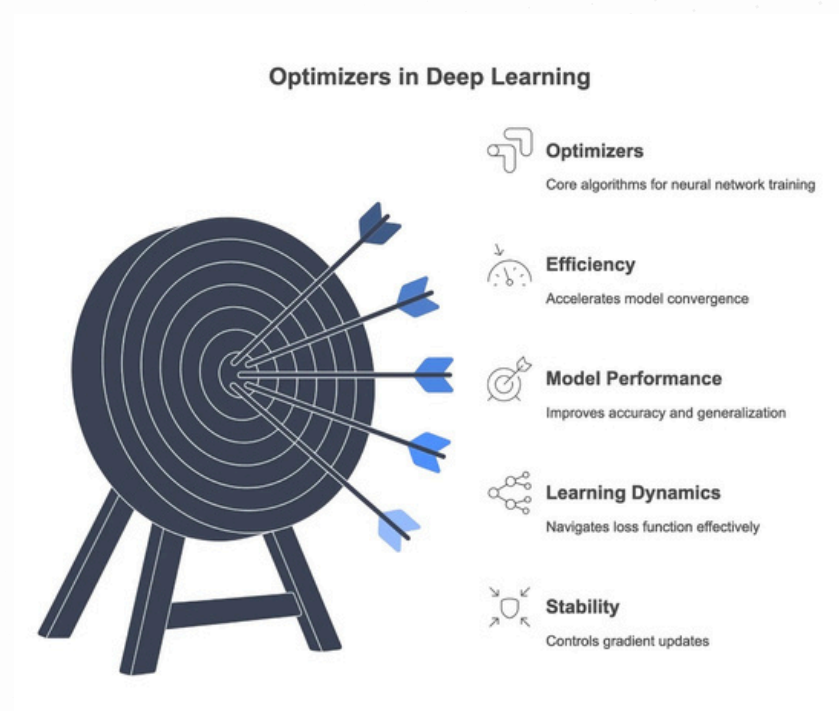


Optimizers in Deep Learning: The Hidden Force That Makes Models Learn



What is Optimizers?

- Optimizers are algorithms that **adjust** neural network parameters (**weights and biases**) to **minimize the loss function** during training
- They determine the direction and magnitude of parameter updates based on computed gradients
- Optimizers **solve the optimization problem** of finding parameter values that **minimize prediction error**.
- The choice of optimizer significantly impacts training speed, convergence, and model performance



Why Do We Need Optimizers in Deep Learning?

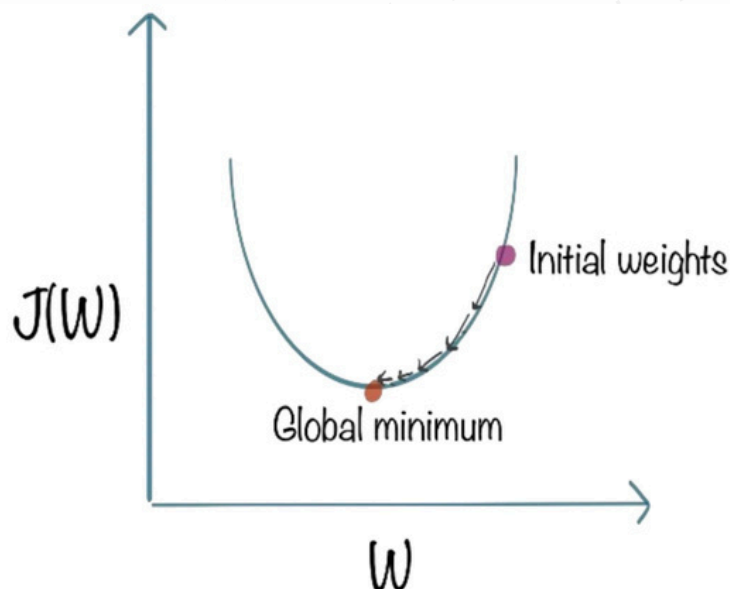
- **Enable Learning:** Neural networks cannot learn without optimizers; they provide the mechanism to adjust weights based on training feedback
- **Navigate Complex Landscapes:** Deep learning loss surfaces are non-convex with multiple local minima and saddle points; optimizers help navigate these terrains
- **Accelerate Convergence:** Efficient optimizers reduce training iterations, saving computational time and resources.
- **Improve Generalization:** Well-chosen optimizers help models find better solutions that generalize well to unseen data



Types of Optimizers

Gradient Descent: The Foundation Stone

- **Update Strategy:** Uses entire training dataset
- **Characteristics:**
 - Guarantees convergence to global minimum for convex functions
 - Stable and smooth parameter updates with predictable behavior
 - Computationally expensive; processes all data before single update
 - Impractical for large datasets and online learning scenarios
- **Best Use Case:** Small to medium datasets where computational cost is manageable and training stability is the top priority.

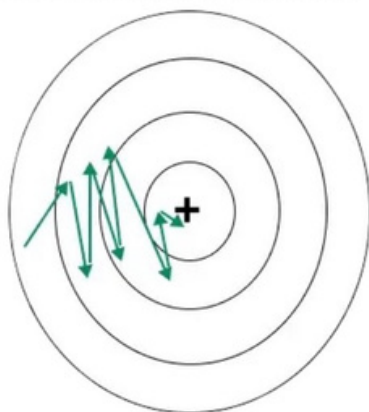


Types of Optimizers

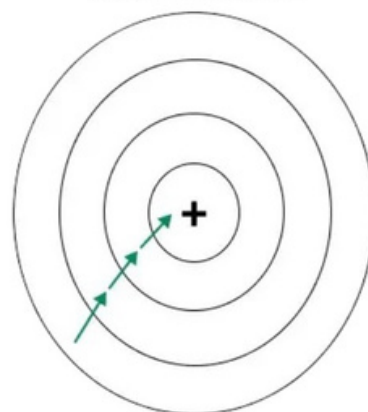
Stochastic Gradient Descent: The Speed Demon

- **Update Strategy:** Uses single random training example
- **Characteristics:**
 - Extremely fast updates with immediate parameter changes
 - Low memory footprint; processes one sample at a time
 - High variance in updates causes noisy, fluctuating convergence
 - Can escape shallow local minima due to stochastic nature
 - Enables online learning with streaming data.
- **Best Use Case:** Large-scale datasets, online learning systems, and scenarios where speed matters more than convergence stability.

Stochastic Gradient Descent



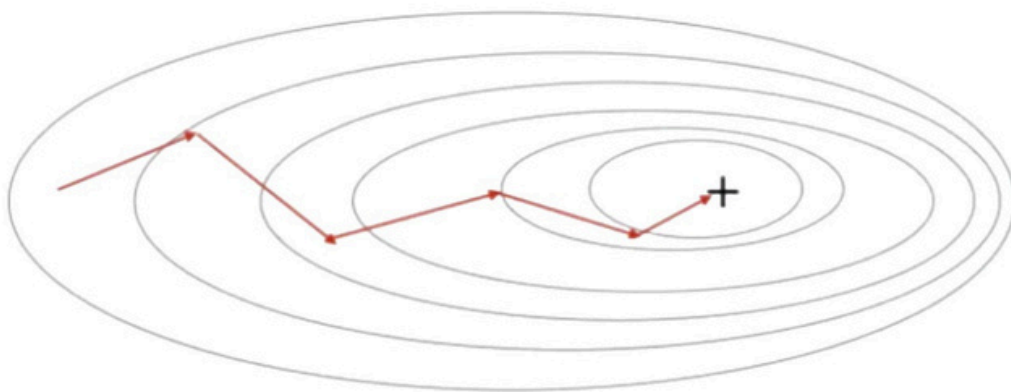
Gradient Descent



Types of Optimizers

Mini-Batch Gradient Descent: The Balanced Performer

- **Update Strategy:** Uses small batches (32, 64, 128, 256 samples)
- **Characteristics:**
 - Perfect balance between batch GD stability and SGD speed
 - Efficiently exploits GPU parallelization and vectorized operations
 - Reduces update variance while maintaining computational efficiency
 - Batch size becomes additional hyperparameter to tune
 - Industry standard for modern deep learning workflows
- **Best Use Case:** Default choice for most deep learning applications, especially when using GPUs or distributed training systems.



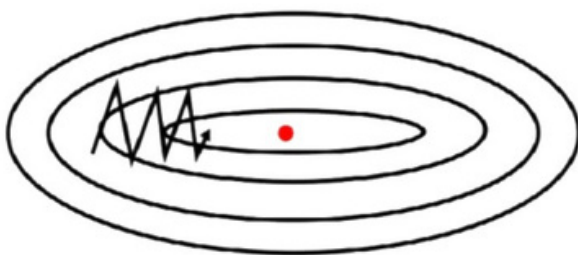
Mini Batch Gradient Descent

Types of Optimizers

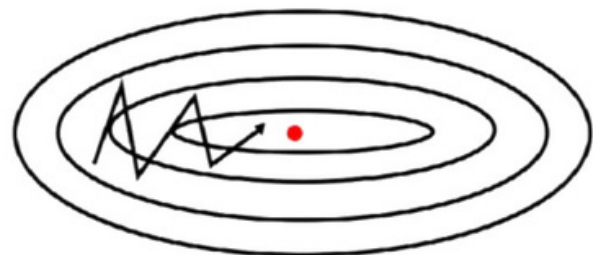
SGD with Momentum: The Accelerator

- **Update Strategy:** Accumulates velocity from previous gradients (β typically 0.9)
- **Characteristics:**
 - Accelerates convergence in directions with consistent gradients
 - Dampens oscillations in high-curvature directions
 - Helps escape small local minima and cross flat plateaus
 - Acts like a ball rolling downhill, building momentum
 - Requires tuning both learning rate and momentum coefficient
- **Best Use Case:** Deep networks with complex loss surfaces, problems with ravines, and when faster convergence is critical.

SGD without momentum



SGD with momentum

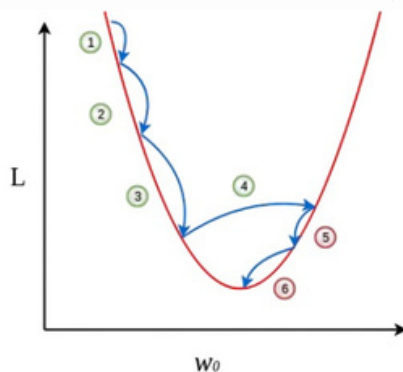


SGD with Momentum

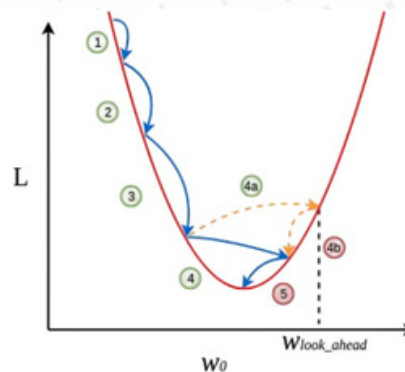
Types of Optimizers

Nesterov Accelerated Gradient: The Look-Ahead Visionary

- **Update Strategy:** Computes gradient at anticipated future position
- **Characteristics:**
 - "Look-ahead" mechanism provides predictive correction
 - More responsive to gradient direction changes than standard momentum
 - Reduces overshooting by evaluating gradient ahead of time
 - Slightly more complex with additional computational overhead
 - Often converges faster than vanilla momentum methods
- **Best Use Case:** Convex optimization problems and scenarios where standard momentum overshoots the optimal solution.



(a) Momentum-Based Gradient Descent



(b) Nesterov Accelerated Gradient Descent

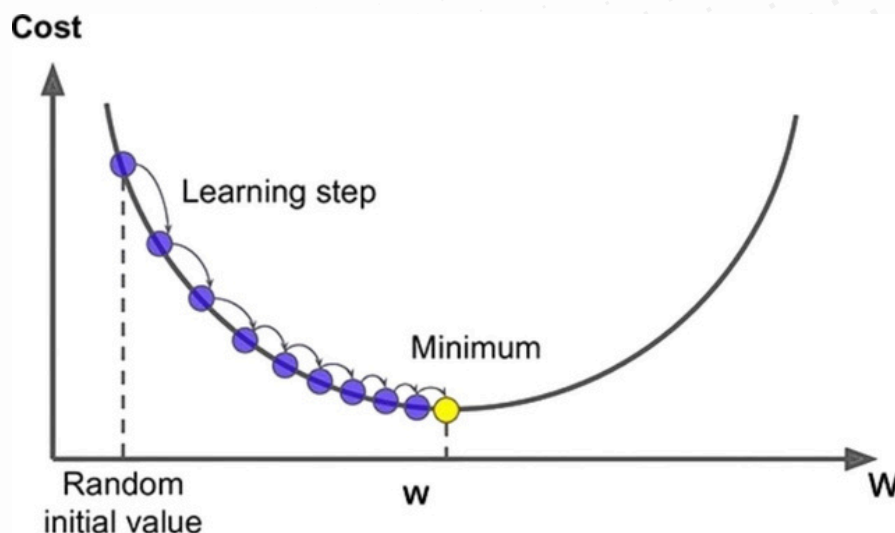
$$\text{Green Circle} \Rightarrow \frac{\partial L}{\partial w_0} = \frac{\text{Negative}(-)}{\text{Positive}(+)}$$

$$\text{Red Circle} \Rightarrow \frac{\partial L}{\partial w_0} = \frac{\text{Negative}(-)}{\text{Negative}(-)}$$

Types of Optimizers

AdaGrad: The Adaptive Pioneer

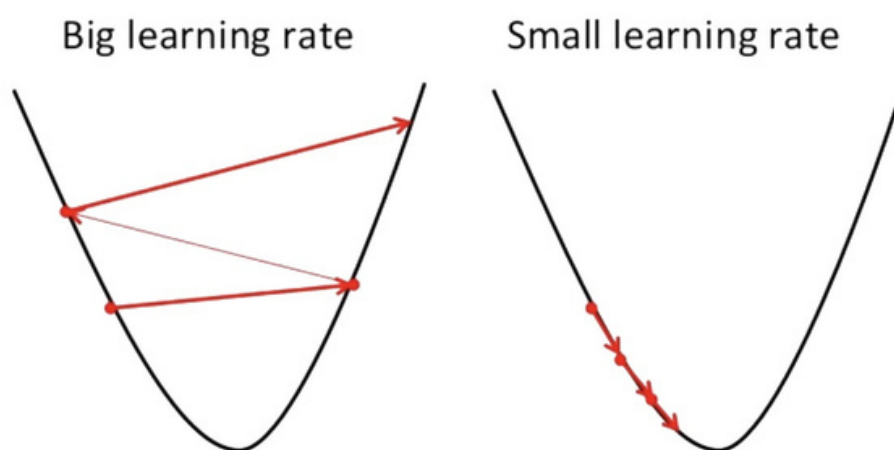
- **Update Strategy:** Adapts learning rate based on cumulative squared gradients
- **Characteristics:**
 - Automatically adjusts learning rate for each parameter individually
 - Excellent for sparse data and features (NLP, embeddings)
 - Learning rate decreases monotonically, often becoming too small
 - Stops learning prematurely before reaching optimal convergence
 - Historical gradient accumulation grows unbounded over time.
- **Best Use Case:** Natural language processing tasks with sparse features, but generally avoid for deep neural networks.



Types of Optimizers

RMSProp: The Decay Master

- **Update Strategy:** Exponentially decaying average of squared gradients (ρ typically 0.9)
- **Characteristics:**
 - Solves AdaGrad's aggressive learning rate decay problem
 - Maintains adaptive properties while allowing continued learning
 - Works effectively with non-stationary objectives
 - Well-suited for recurrent neural networks and time-series
 - Introduced by Geoffrey Hinton, widely adopted before Adam
- **Best Use Case:** Recurrent neural networks, time-series modeling, and problems where AdaGrad's learning rate decays too quickly.



Types of Optimizers

Adam: The Industry Standard

- **Update Strategy:** Combines momentum and RMSProp with bias correction ($\beta_1=0.9$, $\beta_2=0.999$)
- **Characteristics:**
 - Best of both worlds: momentum acceleration + adaptive learning rates
 - Requires minimal hyperparameter tuning with excellent defaults
 - Works reliably across diverse problems and architectures
 - De facto standard in modern deep learning research and production
 - Slightly higher memory usage due to storing two moment estimates
- **Best Use Case:** Default first choice for virtually all deep learning tasks including computer vision, NLP, and reinforcement learning.

How to Choose the Right Optimizer

Quick Decision Guide:

- **Starting Out / General Use:** Adam with default hyperparameters (learning rate = 0.001)
- **Computer Vision (Best Accuracy):** SGD with Momentum (0.9) + learning rate scheduling
- **NLP / Transformers:** Adam (Adam with weight decay regularization)
- **Recurrent Networks:** RMSProp or Adam for handling temporal gradient variations
- **Limited Memory:** SGD or Mini-Batch GD (adaptive methods store additional states)

Key Decision Factors:

- **Dataset Size:** Small → Batch GD; Medium → Mini-Batch GD; Large → SGD/Adam
- **Tuning Resources:** Limited → Adam (robust defaults); Extensive → SGD+Momentum (better peak performance)
- **Training Time:** Quick prototyping → Adam; Production training → SGD+Momentum
- **Generalization Priority:** Maximum → SGD+Momentum; Balanced → Adam/AdamW

Conclusion

- Optimizers are the engine of deep learning, transforming loss gradients into intelligent parameter updates
- No single optimizer dominates all scenarios; choice depends on task, data, and computational constraints
- Adam has emerged as the universal starting point: reliable, efficient, and requiring minimal tuning effort
- SGD with momentum remains the gold standard for state-of-the-art results when generalization matters most
- Success comes from understanding optimizer mechanics, not memorizing rules—experiment intelligently and iterate

Thank You